

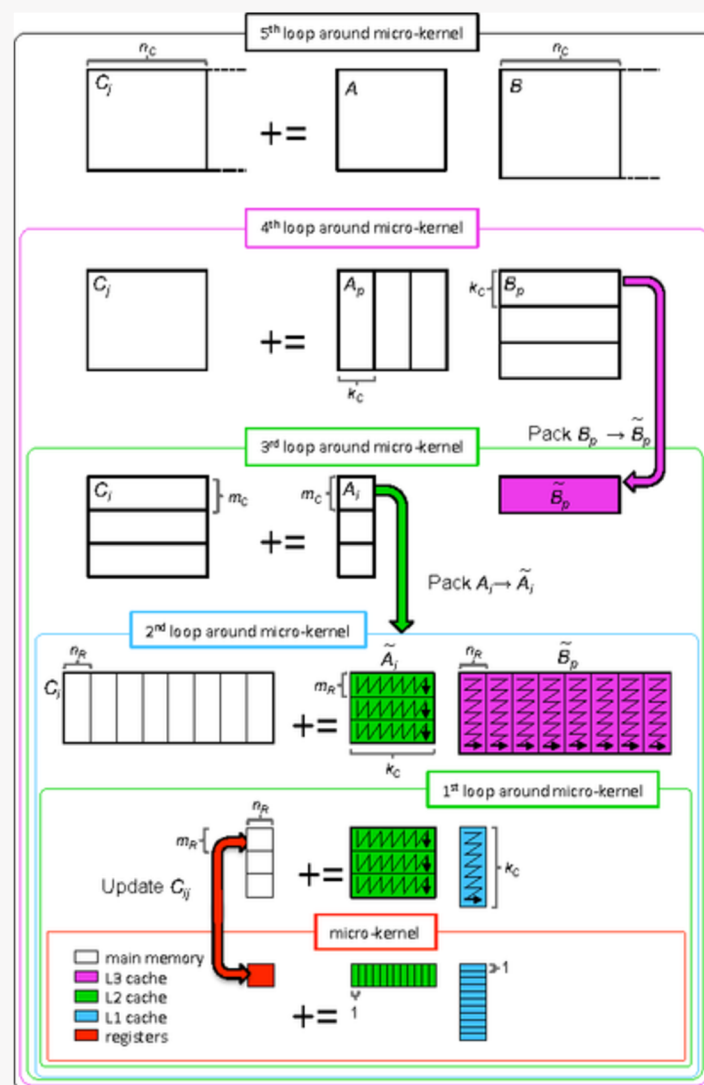
TER

Week 2

Report.

Rosemarie Streefkerk, Edna Iusupova

Core Algorithm (BLIS 5-Loop)



```

Loop 5 for  $j_c = 0 : n-1$  steps of  $n_c$ 
       $J_c = j_c : j_c + n_c - 1$ 
Loop 4 for  $p_c = 0 : k-1$  steps of  $k_c$ 
       $P_c = p_c : p_c + k_c - 1$ 
       $B(P_c, J_c) \rightarrow B_c$  // Pack into  $B_c$ 
Loop 3 for  $i_c = 0 : m-1$  steps of  $m_c$ 
       $I_c = i_c : i_c + m_c - 1$ 
       $A(I_c, P_c) \rightarrow A_c$  // Pack into  $A_c$ 
      // Macro-kernel
Loop 2 for  $j_r = 0 : n_c-1$  steps of  $n_r$ 
       $J_r = j_r : j_r + n_r - 1$ 
Loop 1 for  $i_r = 0 : m_c-1$  steps of  $m_r$ 
       $I_r = i_r : i_r + m_r - 1$ 
      // Micro-kernel
Loop 0 for  $k_r = 0 : k_c-1$ 
       $C_c(I_r, J_r)$ 
       $\leftarrow A_c(I_r, k_r) \ B_c(k_r, J_r)$ 
      +=
    endfor
  endfor
endfor
endfor
endfor

```

- Structured nested loops for cache hierarchy (L3 -> L2 -> Registers)

Improvement:

Transforms strided memory access into contiguous packing.

Loop j_c -> Loop p_c -> Pack B -> Loop i_c -> Pack A ->

Loop j_r -> Loop i_r -> Micro-kernel

Micro-kernels (AVX2 Vect.)

KERNEL	REGISTER USAGE	FLOP/STEP	STATUS
8x8	10/16	128	underutilized
6x16	16/16	192	optimal default
4x24	16/16	192	better for wide N

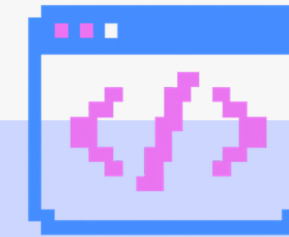
- *6x16 uses the full hardware register budget (16 YMMs).*
- *Be careful with register spills: any extra variable forces data out of the registers.*

Kernel 6×16 ASM

Functionally identical to 6×16 but written in GCC inline x86-64 assembly: bypasses the compiler entirely — guarantees exact register assignment and instruction ordering, Prevents compiler from inserting unneeded spills or reordering FMAs
-> In practice ~5–15% faster than the intrinsics version at large sizes

TASK1 — 2D Parallel

The work is divided across the (ic, jt) tile space — each OpenMP task handles one $MC \times NR$ strip of the output matrix C.



```
#pragma omp for schedule(static) ← parallel B packing  
#pragma omp for schedule(static) ← parallel A packing (all ic at once)  
#pragma omp for collapse(2) schedule(dynamic) ← compute all (ic, jt) tiles
```

Key design decision: A is packed **once** per (pc, ic) combination into a shared **packed_A_all** buffer, not re-packed for every jt strip. This reduced total packing work from **$O(nr_jt \times M \times KC)$** to **$O(M \times KC)$** .

Total tasks: **$\text{ceil}(M/MC) \times \text{ceil}(N/NR)$** . With $MC=120$, $MR=6$ and typical matrices, this gives hundreds of tasks — plenty of parallelism even at 8 threads.

TASK2 — 3D Parallel, K-Repl.

For cases where 2D parallelism isn't enough (small $M \times N$, many threads), the K dimension is split into r independent slices. Each slice computes into a private partial-C buffer, then a reduction merges all partials into the real C .



```
for kr in 0..r:
#pragma omp task firstprivate(...) {
    // compute partial_C[tile][kr] from K-slice [pc_start, pc_end) }
#pragma omp taskwait
    // AVX2 reduction: C[i] = beta*C[i] + sum over kr of partial_C[tile][kr][i]
```

The reduction uses AVX2 FMA to process 8 floats per instruction.

Better core occupancy for non-square "tall" matrices BUT **malloc/free inside OpenMP tasks caused serialization** on the allocator lock → moved outside parallel region.

Autotuning pipeline

The blocking parameters (MC, KC, NC) depend on the machine's actual cache sizes and memory bandwidth. Hard-coding them gives suboptimal results on different hardware. Three auto-tuning algorithms were implemented in Python. All communicate with the C binary via a JSON interface.

The benchmark uses **adaptive-N timing**: it starts with 3 runs, and keeps adding runs until **stddev / median** < **2%** (or hits 15 runs). This gives reliable measurements without wasting time on stable configurations.

1. Gradient descent

Optimizes one parameter at a time (MC, then KC, then NC), cycling until convergence.

Converges in 15–30 evaluations.

Can get stuck in local minima.

2. Bayesian optimization

Uses a Tree-structured Parzen Estimator surrogate model to suggest parameter configurations.

Finds better global optima.

More evaluations needed (30–50)

3. Simulated Annealing

Implemented for comparison purposes. Random search is used to initialize the Bayesian prior.

Results

8 threads

SIZE	OPENBLAS	WEEK 2: 6x16 T1	% OF OPENBLAS
250^3	221.9 GF/s	247.0 GF/s	111%
512^3	252.0 GF/s	384.2 GF/s	152%
1024^3	272.2 GF/s	495.0 GF/s	182%
2048^3	558.2 GF/s	459.3 GF/s	82%

16 threads

SIZE	OPENBLAS	WEEK 2: 6x16 ASM	% OF OPENBLAS
512^3	424.7 GF/s	510.6 GF/s	120%
1024^3	486.8 GF/s	659.3 GF/s	135%
2048^3	517.3 GF/s	781.0 GF/s	151%

Peak absolute performance reached
781 GFLOP/s.

Known limitations

TASK2 Does Not Scale Well

- The reduction step runs sequentially on a single thread after all K-slice tasks complete.
- Partial-C buffers are allocated at $MC \times NC$ size regardless of the actual N-block width (uses more memory than needed and reduces cache effectiveness)
- Makes sense only for cases where $M \times N$ is small and K is very large (thin tall matrices)

Small Matrix Performance (64×64, 128×128)

- all kernels are 20–65% below OpenBLAS (the gap widens at 16T). The blocking overhead (memory allocation, packing) dominates computation for small matrices.

Thread Scaling at 256×256

- At 16 threads, 6×16 TASK1 reaches only 88% of OpenBLAS, whereas at 8T it reached 111%. The tile count at 256×256 with $MC=120$ is only $\text{ceil}(256/120) \times \text{ceil}(256/16) = 2 \times 16 = 32$ tiles: not enough to keep 16 threads fully occupied.

Conclusion + next steps

Successes:

- Architecture-level decisions (packing once, thread-local buffers) delivered target performance.
- Hand-written micro-kernels fully utilize AVX2 hardware.

Future Work:

- Add small-matrix fast path to bypass packing.
- NUMA-aware memory pinning.
- Implement parallel reduction for TASK2 (3D Parallel).